ReachAI: Agentic AI Social Media Planner

Cloud Computing Project

MCA Semester 3

2026-27

A037 Pratham Parmar

A043 Roshith Prakash

A053 Soumyadeep Das

Abstract

ReachAI is a cloud-enabled social media content planning and automation platform designed to simplify the process of transforming video content into platform-specific marketing material. Content creators often need to perform multiple tasks manually, including video analysis, audience identification, search engine optimization, caption generation, scheduling, and performance monitoring. ReachAI addresses these challenges by integrating cloud-based media storage, artificial intelligence services, managed database infrastructure, and asynchronous job processing into a unified web application.

The system uses React and Vite for the frontend, Node.js and Express.js for backend services, MongoDB Atlas for cloud-based data persistence, Cloudinary for media storage and delivery, and Google Gemini for multimodal video analysis and content generation. A modular seven-agent AI pipeline processes video content and generates tailored outputs for YouTube, Instagram, LinkedIn, and X. Firebase Authentication provides user authentication, while Redis supports distributed rate limiting and Agenda.js manages scheduled publishing tasks.

From a cloud computing perspective, the project demonstrates the integration of managed cloud services, cloud-hosted data storage, API-based computing resources, asynchronous processing, and distributed coordination. Direct video uploads to Cloudinary reduce the load on the application server, while Server-Sent Events provide real-time updates during AI processing. Database-backed job coordination helps prevent duplicate publishing when multiple workers attempt to process the same scheduled task. Security measures include encrypted storage of social media OAuth tokens, authenticated API endpoints, and input validation.

ReachAI demonstrates how cloud services can be combined to build a modular and extensible application for automated content management. The project also provides practical experience in cloud resource integration, application security, distributed job execution, and the design of responsive, scalable web applications.

1. INTRODUCTION

1.1 Background

Cloud computing provides on-demand access to computing resources, storage, databases, and application services over the internet. It enables developers to build applications using managed services without maintaining all the underlying infrastructure.

With the growing popularity of platforms such as YouTube, Instagram, LinkedIn, and X, managing video content across multiple platforms has become time-consuming. Content creators must analyze videos, generate captions and hashtags, prepare platform-specific content, schedule posts, and monitor engagement.

ReachAI is a cloud-enabled social media planning and automation platform that simplifies these activities. It integrates Cloudinary for media storage, MongoDB Atlas for data management, Google Gemini for AI-powered video analysis, and Redis for rate limiting. Firebase Authentication manages user identity, while React, Node.js, and Express.js form the application interface and backend.

1.2 Motivation

ReachAI aims to reduce the manual effort involved in managing social media content across multiple platforms. It demonstrates the practical application of cloud computing through managed storage, database services, AI APIs, and asynchronous processing.

The project also focuses on improving efficiency and reliability through real-time progress updates, background job scheduling, secure credential storage, and request rate limiting.

1.3 Problem Statement

Managing video content across multiple platforms requires several repetitive tasks, including video analysis, content generation, SEO optimization, scheduling, and performance monitoring. Handling large video files and coordinating background tasks can also increase server workload and introduce reliability issues.

ReachAI addresses these challenges by integrating cloud services and AI-powered automation into a unified platform for video processing, content generation, draft management, scheduled publishing, and analytics.

1.4 Objectives

The main objectives of ReachAI are:

To develop a cloud-enabled platform for social media content management.

To integrate Google Gemini for video analysis and content generation.

To use Cloudinary for cloud-based media storage and delivery.

To use MongoDB Atlas for persistent application data storage.

To implement a seven-agent AI pipeline for content planning and optimization.

To provide asynchronous processing and real-time progress updates.

To integrate YouTube and Instagram APIs for publishing and analytics.

To improve security through authentication, encryption, input validation, and rate limiting.

To design a modular architecture that supports future extensions.

1.5 Scope of the Project

ReachAI supports video uploads, AI-based analysis, audience recommendations, SEO suggestions, platform-specific content generation, draft editing, scheduling, and performance monitoring.

The application integrates Cloudinary, MongoDB Atlas, Google Gemini, Redis, Firebase Authentication, and Agenda.js to support media storage, data persistence, AI processing, authentication, and background scheduling. Content can be prepared for YouTube, Instagram, LinkedIn, and X, with publishing and analytics integrations for YouTube and Instagram.

The project focuses on cloud service integration, application architecture, data management, asynchronous processing, and security. Additional platforms and cloud capabilities may be incorporated in future versions.

1.6 Significance of the Project

ReachAI demonstrates how cloud services can be integrated to develop a practical social media automation platform. It illustrates the use of managed storage, cloud databases, AI services, asynchronous processing, and distributed task coordination.

The project provides practical experience in cloud service integration, API communication, resource management, application security, and scalable software design.

2. LITERATURE REVIEW

2.1 Cloud Computing

Cloud computing provides on-demand access to computing resources, storage, databases, and software services over the internet. It reduces the need to maintain physical infrastructure and enables applications to integrate managed services. ReachAI uses cloud-based services for media storage, database management, and AI processing.

2.2 Cloud Storage and Database

Cloudinary is used to store and deliver uploaded videos, reducing the need to route large media files through the application server. MongoDB Atlas provides cloud-based data

storage for user profiles, video projects, generated content, and scheduled posts. Prisma ORM facilitates database operations from the backend.

2.3 Artificial Intelligence Services

Google Gemini provides multimodal video analysis and content generation capabilities. ReachAI uses a seven-agent pipeline to analyze videos, identify audiences, generate platform-specific content, optimize SEO, review content quality, and recommend publishing strategies.

2.4 Asynchronous Processing

Long-running AI tasks and scheduled publications require asynchronous execution. ReachAI uses Server-Sent Events (SSE) to display real-time analysis progress and Agenda.js to manage scheduled publishing tasks. MongoDB-backed conditional updates help prevent multiple workers from claiming the same post simultaneously.

2.5 Cloud Security

Cloud-enabled applications require secure authentication, access control, and resource management. ReachAI uses Firebase Authentication, JWT verification, AES-256-CBC encryption for OAuth tokens, input validation, and Redis-backed rate limiting to improve security and control excessive API requests.

2.6 Research Gap

Individual cloud services provide storage, AI processing, and database capabilities, but integrating them into a complete content management workflow presents challenges. ReachAI addresses this by combining these services into a unified platform for video analysis, content generation, scheduling, and performance monitoring.

2.7 Summary

The reviewed technologies form the foundation of ReachAI. Cloudinary, MongoDB Atlas, Google Gemini, Redis, and Agenda.js work together to support media storage, data management, AI processing, request control, and scheduled publishing.

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The following hardware is sufficient for development and testing of ReachAI:

- **Processor:** Intel Core i5 or equivalent
- **RAM:** Minimum 8 GB; 16 GB recommended

- **Storage:** Sufficient space for the source code, dependencies, and temporary files
- **Internet:** Stable internet connection for cloud services and external APIs

Since ReachAI uses cloud-based services for media storage, database management, and AI processing, local hardware requirements are relatively modest.

3.2 Software Requirements

The project uses the following software and cloud services:

- **Frontend:** React 18, Vite, HTML, CSS, JavaScript
- **Backend:** Node.js and Express.js
- **Database:** MongoDB Atlas with Prisma ORM
- **Cloud Storage:** Cloudinary
- **AI Service:** Google Gemini API
- **Authentication:** Firebase Authentication
- **Rate Limiting:** Redis
- **Background Jobs:** Agenda.js
- **External APIs:** YouTube Data API v3 and Meta Graph API
- **Development Tools:** Visual Studio Code, Git, and GitHub

3.3 Functional Requirements

The main functional requirements of ReachAI are:

1. **User Authentication:** Allow users to register and securely access the application.
2. **Video Upload:** Enable users to upload videos for processing and analysis.
3. **AI Video Analysis:** Analyze video content using Google Gemini.
4. **Content Generation:** Generate captions, hashtags, SEO suggestions, and platform-specific content.
5. **Content Management:** Allow users to review, edit, and save generated drafts.
6. **Post Scheduling:** Schedule content for publishing through background jobs.
7. **Social Media Integration:** Connect supported YouTube and Instagram accounts.
8. **Analytics:** Display available publishing and engagement information.
9. **Progress Tracking:** Provide real-time updates during AI processing.

10. **Secure Data Management:** Store application data and protect connected-account credentials.

3.4 Non-Functional Requirements

- **Scalability:** Use managed cloud services that can support increasing data and processing requirements.
- **Performance:** Process tasks asynchronously where appropriate and provide progress updates.
- **Security:** Apply authentication, input validation, encrypted token storage, and rate limiting.
- **Reliability:** Use persistent storage and background job scheduling to manage application operations.
- **Maintainability:** Organize the application into modular frontend, backend, and AI components.
- **Usability:** Provide a simple interface for uploading videos, editing content, scheduling posts, and viewing analytics.
- **Interoperability:** Integrate with external AI and social media APIs.

3.5 Summary

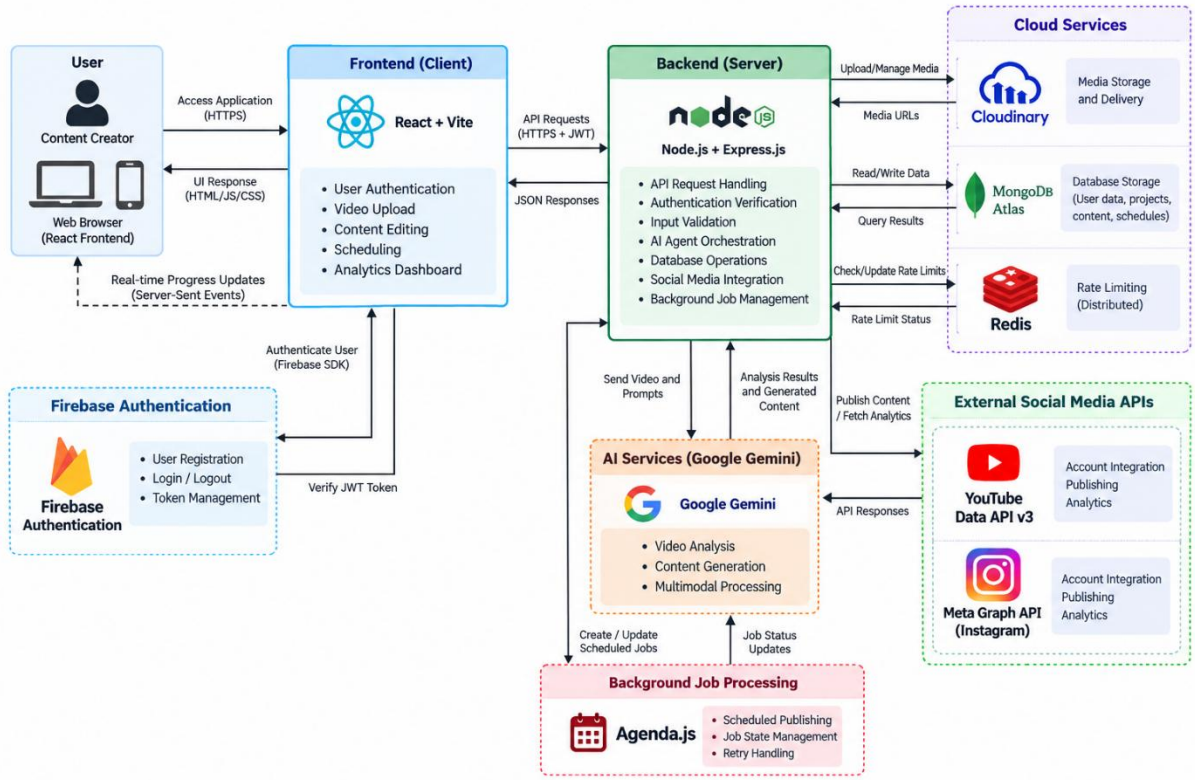
ReachAI combines a web application architecture with cloud-based storage, database, authentication, and AI services. These requirements support its core functions while emphasizing security, maintainability, asynchronous processing, and integration with external platforms.

4. SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture

ReachAI follows a modular client-server architecture that integrates cloud services and external APIs. The React frontend provides the user interface, while the Node.js and Express.js backend handles requests, authentication, AI orchestration, and application logic.

Cloudinary manages media storage and delivery, MongoDB Atlas stores application data, Google Gemini performs AI-powered video analysis and content generation, and Redis supports rate limiting. Firebase Authentication manages user identity, while Agenda.js handles scheduled background jobs.



4.2 Major Components

1. Frontend: Developed using React and Vite, it provides video uploads, content editing, scheduling, progress tracking, and analytics interfaces.

2. Backend: Node.js and Express.js process API requests, validate inputs, coordinate AI agents, and communicate with databases and external services.

3. Cloud Storage: Cloudinary stores uploaded media and provides media delivery capabilities.

4. Cloud Database: MongoDB Atlas stores user-related information, projects, generated content, and scheduled posts. Prisma ORM supports database operations.

5. AI Processing: Google Gemini powers a seven-agent pipeline that analyzes videos and generates content recommendations.

6. Background Processing: Agenda.js manages scheduled publishing jobs, while Redis provides distributed rate limiting.

7. External Integrations: YouTube Data API v3 and Meta Graph API support the implemented social account integration, publishing, and analytics functionality.

4.3 AI Agent Pipeline

ReachAI uses seven specialized agents to organize the content generation process:

1. **VideoAnalysisAgent:** Analyzes the uploaded video.

2. **AudienceAgent:** Develops audience-related recommendations.
3. **PlatformAgent:** Adapts recommendations for target platforms.
4. **SEOAgent:** Generates SEO-related suggestions and hashtags.
5. **ContentAgent:** Produces captions and other content.
6. **QualityAgent:** Reviews the generated output.
7. **StrategyAgent:** Provides overall content strategy recommendations.

The pipeline divides content generation into specialized tasks, making the process more structured and modular.

4.4 Data Flow

ReachAI uses a direct-to-cloud approach for video uploads to reduce backend bandwidth and memory overhead.

1. The user authenticates through Firebase Authentication and accesses the React frontend.
2. The frontend uploads the video binary directly to Cloudinary using signed upload parameters, bypassing the Express backend during the initial upload.
3. Cloudinary returns the media URL and metadata, which the frontend sends to the backend for project registration in MongoDB Atlas.
4. The backend retrieves the video from Cloudinary and submits it to the Google Gemini File API for multimodal analysis.
5. The seven-agent pipeline generates video insights, audience recommendations, SEO suggestions, platform-specific content, quality checks, and scheduling strategies.
6. The backend persists the results in MongoDB Atlas and sends processing updates to the frontend through Server-Sent Events (SSE).
7. Agenda.js manages scheduled publishing jobs and communicates with the supported social media APIs.
8. Available publishing and engagement data is displayed on the analytics dashboard.

Server-Sent Events (SSE) provide real-time progress updates to the frontend during supported processing operations.

4.5 Cloud Computing Approach

ReachAI uses a combination of managed cloud services and external APIs rather than relying entirely on local infrastructure. Cloudinary handles media storage, MongoDB Atlas provides a managed database, and Google Gemini supplies AI processing capabilities. Firebase Authentication and Redis support authentication and rate limiting.

This approach reduces the need to manage every infrastructure component independently. It also allows individual services to be maintained or scaled according to their requirements.

Service Model	**Technology**	**Role in ReachAI**
Database as a Service (DBaaS)	MongoDB Atlas	Managed database for user profiles, projects, generated content, and scheduled posts.
AI as a Service (AIaaS)	Google Gemini API	Multimodal video analysis and AI-powered content generation.
Identity as a Service (IDaaS)	Firebase Authentication	User authentication and identity management.
Cloud Media Storage and Delivery	Cloudinary	Video storage, media processing, and CDN delivery.
Managed In-Memory Datastore	Redis / Upstash	Distributed rate limiting and request control.
Application Runtime	Node.js and Express.js	Backend request handling, AI orchestration, and application logic.
Frontend Application	React and Vite	User interface for uploads, content editing, scheduling, and analytics.

The application combines services from multiple providers, forming a multi-cloud service architecture. This allows ReachAI to use specialized services for media management, database storage, authentication, and AI processing.

4.6 Security and Reliability

The system uses Firebase Authentication and backend JWT verification to authenticate requests. Input validation, Helmet, CORS configuration, and Redis-based rate limiting provide additional safeguards. OAuth tokens are encrypted before storage.

Scheduled jobs use database state transitions to reduce the possibility of multiple workers claiming the same post. However, failures during communication with an

external social media platform can still result in uncertain publishing outcomes, so the system should not be considered to guarantee exactly-once delivery.

4.7 Summary

ReachAI combines a modular web application with cloud storage, managed database services, AI processing, and background scheduling. Its architecture separates user interaction, application logic, media management, AI orchestration, and publishing tasks, supporting maintainability and future extension.

5. IMPLEMENTATION DETAILS

5.1 Frontend Implementation

The frontend of ReachAI is developed using React 18 and Vite. It provides interfaces for user authentication, video uploads, AI processing progress, content editing, post scheduling, and analytics. Server-Sent Events (SSE) are used to display real-time progress updates during supported processing operations.

5.2 Backend Implementation

The backend is developed using Node.js and Express.js. It manages API requests, validates input, verifies authentication tokens, coordinates AI agents, interacts with cloud services, and handles scheduling operations. The modular backend structure separates application logic from external service integrations.

5.3 Cloud Media Storage

Cloudinary is used to store and deliver uploaded media. This reduces the need to route media storage and delivery entirely through the application server. The backend coordinates uploads and processing while Cloudinary provides the associated cloud media capabilities.

5.4 Database Management

MongoDB Atlas stores application information, including project data, generated content, and scheduled posts. Prisma ORM is used to support database operations. Persistent storage allows users to access saved content and scheduled publishing information across sessions.

5.5 AI Content Processing

Google Gemini is integrated for multimodal video analysis and content generation. The seven-agent pipeline divides the workflow into video analysis, audience recommendations, platform adaptation, SEO optimization, content generation, quality

review, and strategy recommendations. This structured approach supports the generation of content tailored to different social media platforms.

5.6 Background Job Scheduling

Agenda.js manages scheduled publishing tasks. The backend stores scheduling information and processes jobs when they become due. Database state transitions help prevent multiple workers from claiming the same post simultaneously. External API failures may still require retry handling and status reconciliation.

5.7 Social Media Integration

ReachAI integrates with YouTube Data API v3 and Meta Graph API for supported YouTube and Instagram account operations. These integrations enable applicable publishing and analytics features, subject to platform permissions, account requirements, and API limitations.

5.8 Security Implementation

The application uses Firebase Authentication and JWT verification for access control. OAuth tokens are encrypted before being stored, while Zod validation, Helmet, CORS configuration, and Redis-based rate limiting provide additional security measures. These mechanisms help protect user data and control access to application endpoints.

5.9 Summary

The implementation combines a React frontend, Node.js backend, cloud media storage, managed database services, AI processing, and background scheduling. These components work together to provide an integrated workflow for video analysis, content generation, scheduling, publishing, and analytics.

6. TESTING AND RESULTS

6.1 Testing Overview

Testing is performed to verify the functionality and integration of ReachAI's frontend, backend, database, AI services, and cloud components. The main focus is on validating application workflows, data handling, security controls, and communication with external APIs.

6.2 Functional Testing

Functional testing covers the application's core features, including:

- User authentication and access control.

- Video upload and cloud media storage.
- AI-powered video analysis and content generation.
- Editing and saving generated content.
- Scheduling posts through background jobs.
- Integration with supported social media APIs.
- Displaying processing progress and available analytics.

6.3 Cloud Service Integration Testing

Cloud integration testing verifies communication between the application and its external services.

- **Cloudinary:** Checks media upload and retrieval workflows.
- **MongoDB Atlas:** Verifies data storage and retrieval.
- **Google Gemini:** Checks AI analysis and content generation requests.
- **Firebase Authentication:** Verifies authentication and token validation.
- **Redis:** Checks rate-limiting functionality.
- **Social Media APIs:** Tests supported account integration and publishing operations.

6.4 Security Testing

Security testing focuses on authentication, input validation, encrypted OAuth token storage, CORS configuration, security headers, and rate limiting. These checks help identify unauthorized access and invalid requests. Encryption key management and protection against token exposure are also important considerations.

6.5 Background Processing Testing

Background processing tests examine scheduled job execution, post status transitions, and handling of API errors. Database state transitions help reduce duplicate job claims. However, external publishing failures and network interruptions must also be considered when validating reliability.

6.6 Results and Observations

ReachAI brings together cloud-based media storage, database management, AI processing, authentication, and background scheduling in a unified application. Its modular architecture separates major responsibilities, while SSE supports real-time progress updates and Agenda.js coordinates scheduled tasks.

The application demonstrates how managed cloud services and external APIs can be combined to automate several stages of social media content management. Actual performance, reliability, and scalability should be evaluated using measured test results under representative workloads.

6.7 Summary

Testing evaluates the correctness of the application's main workflows and the integration of its cloud services. The results help assess functionality, security, and reliability while identifying areas for further optimization.

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

ReachAI demonstrates the integration of cloud computing and artificial intelligence to simplify social media content management. The application combines Cloudinary for media storage, MongoDB Atlas for database management, Google Gemini for AI-powered video analysis and content generation, and Firebase Authentication for user identity management. Redis supports rate limiting, while Agenda.js manages scheduled background tasks.

The seven-agent AI pipeline supports video analysis, audience recommendations, SEO optimization, platform-specific content generation, quality review, and content strategy. Real-time progress updates and social media API integrations further support the content management workflow. Overall, ReachAI illustrates how managed cloud services and external APIs can be combined to build a modular application with reduced dependence on locally managed infrastructure.

7.2 Future Scope

The following improvements can be considered for future development:

1. **Additional Platform Integrations:** Extend publishing and analytics support to more social media platforms.
2. **Improved AI Recommendations:** Use engagement data to refine content suggestions and platform-specific strategies.
3. **Enhanced Scalability:** Introduce additional worker capacity and workload management for larger processing volumes.
4. **Advanced Analytics:** Provide deeper insights into audience engagement and content performance.
5. **Improved Reliability:** Strengthen retry handling, failure recovery, and reconciliation for scheduled publishing.

6. **Security Enhancements:** Adopt authenticated encryption for stored credentials and improve key management.
7. **Performance Optimization:** Evaluate processing latency, API response times, and resource utilization under different workloads.

7.3 Final Remarks

ReachAI provides a practical demonstration of cloud service integration for AI-assisted social media management. Its architecture combines storage, database, authentication, AI processing, and background scheduling into a unified workflow. Further testing and optimization can help improve its reliability, security, and ability to support larger workloads.

8. REFERENCES

1. National Institute of Standards and Technology (NIST), *The NIST Definition of Cloud Computing*, Special Publication 800-145, 2011.
2. Cloudinary, *Cloudinary Documentation*. Available at: https://cloudinary.com/documentation
3. MongoDB, *MongoDB Atlas Documentation*. Available at: https://www.mongodb.com/docs/atlas/
4. Google AI for Developers, *Gemini API Documentation*. Available at: https://ai.google.dev/gemini-api/docs
5. Firebase, *Firebase Authentication Documentation*. Available at: https://firebase.google.com/docs/auth
6. Redis, *Redis Documentation*. Available at: https://redis.io/docs/latest/
7. Agenda.js, *Agenda: A Job Scheduling Library for Node.js*. Available at: https://github.com/agenda/agenda
8. Meta for Developers, *Instagram Platform Documentation*. Available at: https://developers.facebook.com/docs/instagram-platform/
9. Google for Developers, *YouTube Data API Documentation*. Available at: https://developers.google.com/youtube/v3
10. Express.js, *Express Web Framework Documentation*. Available at: https://expressjs.com/
11. Prisma, *Prisma ORM Documentation*. Available at: https://www.prisma.io/docs
12. Node.js, *Node.js Documentation*. Available at: https://nodejs.org/docs/